

Industrial Internship Report on

Forecasting of Smart City Traffic Patterns using Machine Learning

An Internship Report submitted in partial fulfilment of the
Industrial Internship Program by upskill Campus (USC) in collaboration with UniConverge Technologies Pvt.
Ltd. (UCT)

Prepared by
[Shalini kumari]
[IILM UNIVERSITY GREATER NOIDA]

Domain: Smart City | Project 9
upskill Campus • The IoT Academy • UniConverge Technologies Pvt. Ltd.

Executive Summary

This report presents the work carried out during the Industrial Internship offered by upskill Campus and The IoT Academy, in collaboration with the industry partner UniConverge Technologies Pvt. Ltd. (UCT). The internship was centred on a project assigned by UCT, to be completed along with this report within a period of six weeks.

The assigned project, titled “Forecasting of Smart City Traffic Patterns using Machine Learning,” falls under the Smart City domain. The objective was to analyse historical traffic data collected at four road junctions of a city and to build a machine learning model capable of predicting the number of vehicles passing through a junction at a given point in time. Such a system can help city administrators anticipate traffic peaks and plan infrastructure and traffic-management decisions more effectively.

The work involved loading and exploring the provided dataset, cleaning and pre-processing the data, engineering time-based features from the DateTime column, visualising traffic trends and correlations between variables, training a Linear Regression model, evaluating its performance using Mean Absolute Error (MAE) and Mean Squared Error (MSE), saving the trained model using Joblib, and finally building a simple command-line interface that accepts user input and returns a traffic prediction.

This internship provided practical, hands-on exposure to a complete, end-to-end machine learning workflow — from raw data to a deployable prediction script — and was an overall valuable and enriching experience.

Table of Contents

Contents

Executive Summary.....	2
Table of Contents.....	3
1. Preface	4
2. Introduction	4
2.1 About UniConverge Technologies Pvt. Ltd.	4
2.2 About upskill Campus (USC).....	5
The IoT Academy.....	5
2.3 Objectives of this Internship Program	5
2.4 References	6
2.5 Glossary.....	6
3. Problem Statement.....	7
4. Existing and Proposed Solution.....	7
4.1 Existing Solution	7
4.2 Proposed Solution.....	7
5. Proposed Design / Model	8
5.1 High-Level Diagram	8
5.2 Low-Level Diagram.....	9
6. Performance Test.....	11
6.1 Test Plan / Test Cases.....	11
6.2 Test Procedure	11
6.3 Performance Outcome.....	12
7. My Learnings.....	13
8. Future Work Scope	13
9. Project Screenshots.....	14
9.1 Terminal Output.....	14
9.2 Data Preprocessing	15
9.3 Correlation Heatmap	16
9.4 Average Traffic at Junctions.....	16
9.5 Model Evaluation Output (MAE & MSE).....	17
9.6 Model Saved.....	17
9.7 Final Prediction Output.....	17
10. GitHub Repository.....	19
11. Skills Used	19

1. Preface

This report summarizes the six weeks of work carried out as part of the Industrial Internship Program conducted by upskill Campus (USC) in association with The IoT Academy and industry partner UniConverge Technologies Pvt. Ltd. (UCT).

Industrial internships play an important role in bridging the gap between classroom learning and real-world engineering practice. They provide an opportunity to work on problems that industries actually face, to apply theoretical concepts to practical data, and to develop the discipline required to plan and deliver a project within a fixed timeline.

For this internship, I was assigned a project from the Smart City domain — forecasting traffic patterns at city junctions using machine learning. The program was structured so that the project was broken down into smaller, sequential milestones: understanding the problem statement, exploring the dataset, pre-processing the data, building visualisations, training a model, evaluating it, and finally packaging the trained model into a usable prediction script.

Working through this internship helped me strengthen my practical skills in Python, Pandas, data visualisation with Matplotlib, and building a regression model with scikit-learn. It also helped me understand how a real machine learning pipeline is structured — starting from raw CSV data and ending with a saved, reusable model.

I would like to thank Upskill Campus (USC), The IoT Academy, and UniConverge Technologies Pvt. Ltd. (UCT) for providing me with this valuable internship opportunity and the resources to successfully complete this project.

To juniors and peers who will take up this internship in the future: start with understanding the dataset thoroughly before writing any model code, and keep your pre-processing and modelling steps consistent with each other — it will save you a lot of debugging time later.

2. Introduction

2.1 About UniConverge Technologies Pvt. Ltd.

UniConverge Technologies Pvt. Ltd. (UCT) is a company established in 2013, working in the Digital Transformation domain and providing industrial solutions with a focus on sustainability and return on investment (RoI).

To build its products and solutions, UCT leverages a range of cutting-edge technologies, including the Internet of Things (IoT), Cyber Security, Cloud Computing (AWS, Azure), Machine Learning, communication technologies (4G/5G/LoRaWAN), Java full-stack development, Python, and front-end technologies.

- UCT IoT Platform (UCT Insight) — a platform for quick deployment of IoT applications, built on a Java backend and ReactJS front end, with support for MySQL and various NoSQL databases. It supports device connectivity through MQTT, CoAP, HTTP, and Modbus TCP/OPC UA, and offers dashboards, analytics, alerting, third-party integration (Power BI, SAP, ERP), and a rule engine.
- Smart Factory Platform (Factory Watch) — a platform for smart-factory needs, offering a scalable solution for production and asset monitoring, OEE and predictive-maintenance capability, and a modular architecture that lets users scale from a single service to more complex solutions.
- LoRaWAN-based Solutions — UCT is an early adopter of LoRaWAN technology, providing solutions for Agritech, Smart Cities, Industrial Monitoring, and Smart Street Lighting / Water / Gas / Electricity metering.
- Predictive Maintenance — UCT provides industrial machine health monitoring and predictive-maintenance solutions using embedded systems, industrial IoT, and machine learning to estimate the remaining useful life of production machinery.

2.2 About upskill Campus (USC)

upskill Campus, together with The IoT Academy and in association with UniConverge Technologies, facilitated the smooth execution of this internship program. USC is a career-development platform that delivers personalised, executive-style coaching in a way that is more affordable, scalable, and measurable than traditional training programs.

The IoT Academy

The IoT Academy is the EdTech division of UCT, running long-term executive certification programs in collaboration with the EICT Academy and IIT Kanpur, IIT Roorkee, and IIT Guwahati across multiple technology domains.

2.3 Objectives of this Internship Program

The objectives of this internship program were to:

- Gain practical experience of working on an industry-assigned problem.
- Apply machine learning concepts to solve a real-world, Smart-City traffic-forecasting problem.
- Improve job prospects through hands-on exposure to an industrial project workflow.
- Deepen understanding of data science and machine learning and their real-world applications.
- Support personal growth in areas such as communication, planning, and problem solving.

2.4 References

- [1] Pandas Documentation — <https://pandas.pydata.org/docs/>
- [2] scikit-learn Documentation — <https://scikit-learn.org/stable/documentation.html>
- [3] Matplotlib Documentation — <https://matplotlib.org/stable/index.html>
- [4] Joblib Documentation — <https://joblib.readthedocs.io/>
- [5] Project dataset — “Smart City Traffic Patterns” dataset provided as part of the upskill Campus / UCT Machine Learning Internship project list (train_aWnotuB.csv, test dataset).

2.5 Glossary

Term	Meaning
DateTime	Timestamp of a traffic reading, later split into Year, Month, Day and Hour.
Junction	Identifier (1–4) of the road junction at which vehicle count was recorded.
Vehicles	Target variable — number of vehicles recorded at a junction in an hour.
ID	Unique record identifier derived from date, time and junction number.
MAE	Mean Absolute Error — average of the absolute differences between predicted and actual values.
MSE	Mean Squared Error — average of the squared differences between predicted and actual values.
Linear Regression	A supervised learning algorithm that models the target as a linear function of the input features.
Joblib	A Python library used to serialise (save) and load trained machine learning models.
Correlation Heatmap	A colour-coded matrix visualisation showing the pairwise correlation between numeric columns.

3. Problem Statement

Many city governments are working towards transforming their cities into “Smart Cities” — digitally enabled cities that provide more efficient services to citizens. Traffic congestion is one of the recurring problems faced in this transformation.

For this project, historical hourly traffic-count data recorded at four road junctions of a city (from November 2015 to June 2017) was provided. The task was to study this data and build a model that could forecast the traffic volume (number of vehicles) at a junction for a given point in time, so that the city administration could better anticipate traffic peaks and plan road infrastructure and traffic management ahead of time.

A separate test dataset (without the Vehicles column) was also provided, representing future junction–time combinations for which traffic volume needed to be predicted.

4. Existing and Proposed Solution

4.1 Existing Solution

Traditionally, traffic management in most cities relies on manual observation, fixed-time traffic signals, and simple statistical averages (such as the historical average traffic for a time slot) to anticipate congestion. Some cities use sensor-based counters or camera feeds combined with basic threshold-based alerts. These approaches are limited because they do not learn from the underlying patterns in historical data and cannot adapt well to how traffic volume changes across junctions, hours of the day, or over time.

4.2 Proposed Solution

The proposed solution uses the historical hourly traffic records available for each junction to train a machine learning model — specifically a Linear Regression model — that learns the relationship between a junction's identity, its record identifier, and the resulting traffic volume. Once trained, the model can be reused (through a saved .pkl file) to generate traffic-volume predictions for new junction and time combinations without needing to be retrained each time.

The value addition of this approach over the existing solution is that predictions are derived directly from historical patterns present in the data rather than fixed rules, the trained model can be saved and reused instantly through Joblib, and the same pipeline can be extended in the future to include more informative time-based features (Year, Month, Day, Hour) that were engineered during pre-processing but not yet included in the final regression model used for prediction.

5. Proposed Design / Model

The solution was implemented as a sequence of Python scripts (Step 1 to Step 12), each responsible for one stage of the machine learning pipeline — from loading the raw CSV files to serving predictions from a saved model. The diagrams below summarise the design at three levels of detail.

5.1 High-Level Diagram

High-Level Diagram of the Smart City Traffic Prediction System

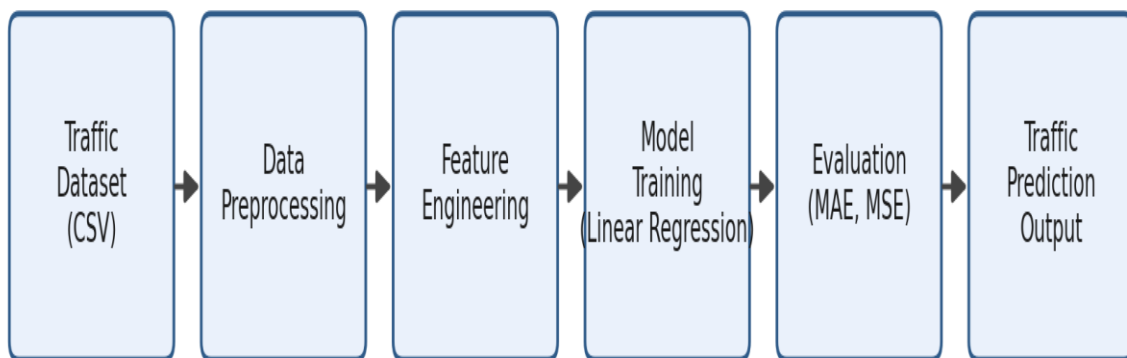


Figure 1: High-level diagram of the Smart City Traffic Prediction system.

5.2 Low-Level Diagram

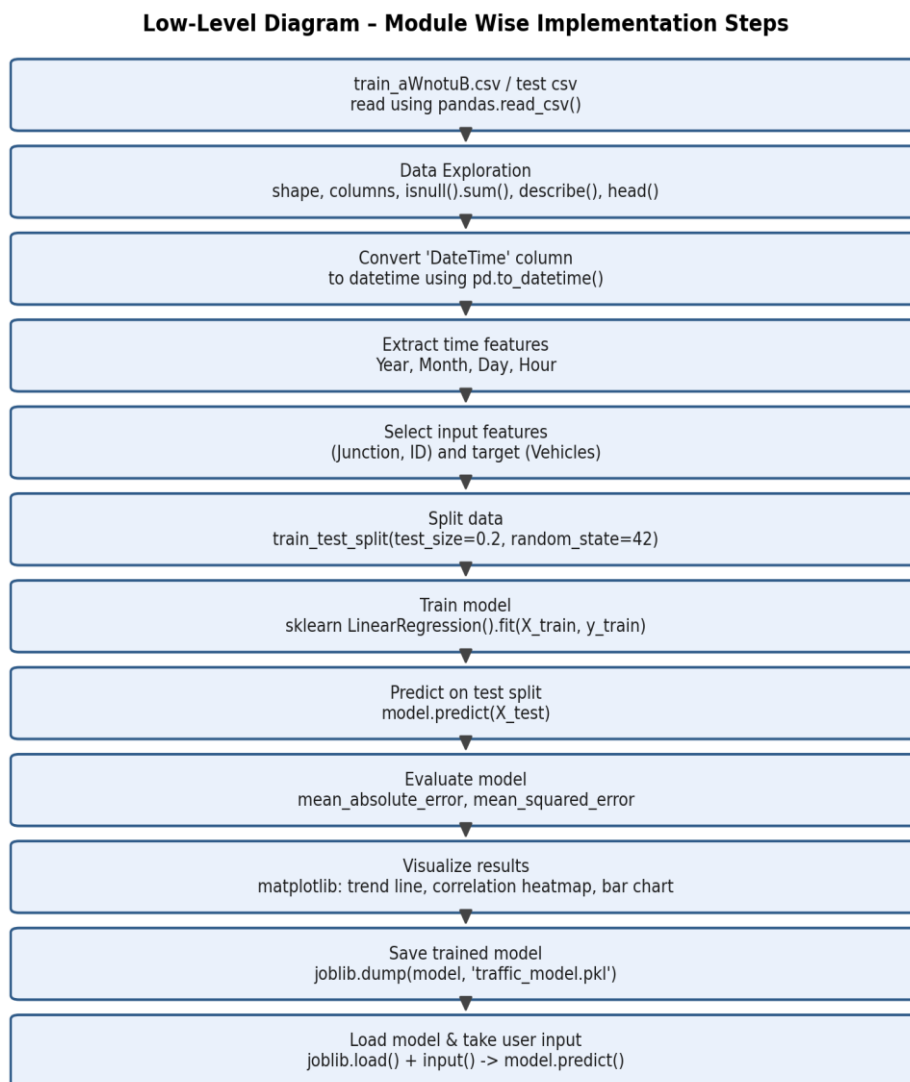


Figure 2: Low-level, module-wise breakdown of the implementation (Step 1 – Step 12).

5.3 System Workflow / Flowchart

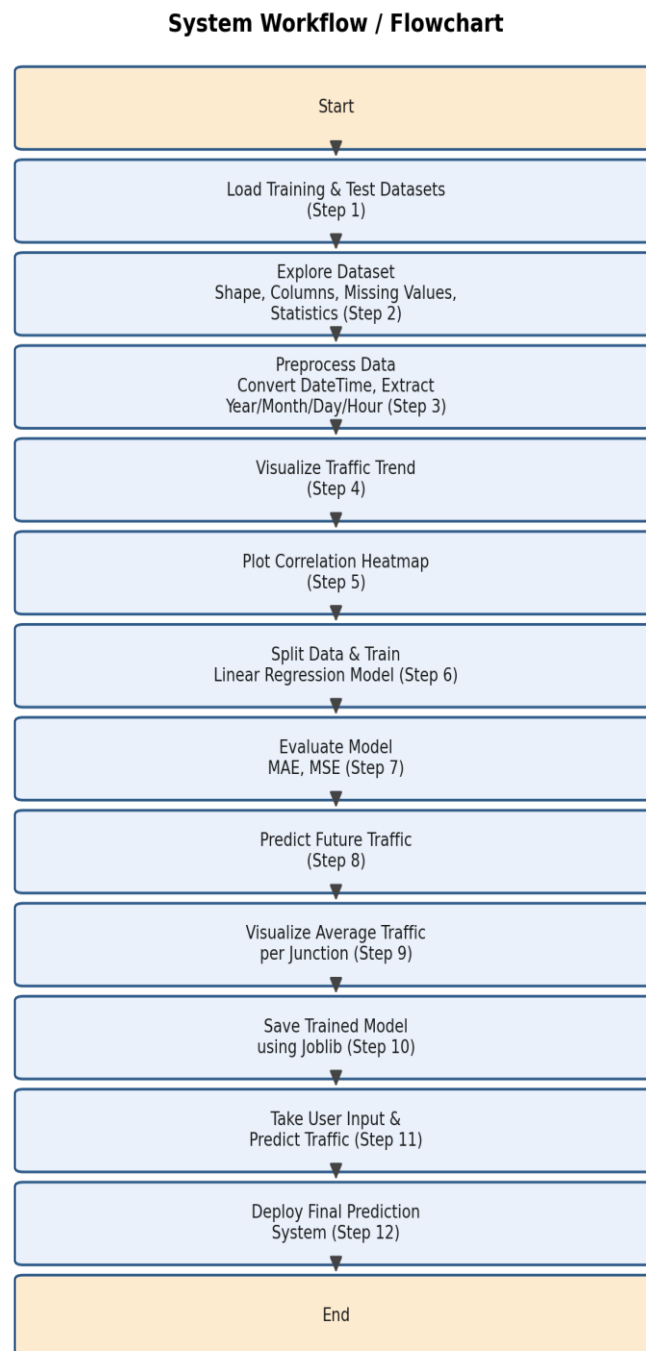


Figure 3: System workflow / flowchart of the complete pipeline.

6. Performance Test

Since this is a data-driven, machine-learning based system, its “performance” was assessed in terms of prediction accuracy on unseen data rather than hardware constraints such as memory or speed. The main constraint identified was the limited set of input features actually available to the model at prediction time — the final saved model uses only the Junction number and the record ID, since the future test dataset does not carry a ready-made Year/Month/Day/Hour breakdown for every record in the exact format the model was trained on.

6.1 Test Plan / Test Cases

The dataset (48,120 hourly records across 4 junctions, from 1 Nov 2015 to 30 Jun 2017) was split into a training set and a hold-out test set using an 80/20 split (`test_size = 0.2`, `random_state = 42`), so that the model could be evaluated on data it had not seen during training.

- Test Case 1: Verify the dataset loads correctly and has no missing values (checked using `isnull().sum()`).
- Test Case 2: Verify the DateTime column converts correctly and Year/Month/Day/Hour features can be extracted without errors.
- Test Case 3: Train the Linear Regression model on the training split and confirm it fits without error.
- Test Case 4: Evaluate the trained model on the 20% hold-out split using MAE and MSE.
- Test Case 5: Save the trained model with Joblib, reload it, and confirm that predictions from the reloaded model match the original model.
- Test Case 6: Provide a new Junction/ID combination through the command-line interface and confirm a numeric traffic prediction is returned.

6.2 Test Procedure

Data exploration confirmed the dataset contains 48,120 rows and 4 columns (DateTime, Junction, Vehicles, ID) with zero missing values in any column. The DateTime column was successfully parsed and used to derive Year, Month, Day and Hour features during pre-processing (Step 3). A correlation heatmap (Step 5) was generated to study relationships between the numeric columns, and a Linear Regression model was trained (Step 6) using an 80/20 train-test split. The model was then evaluated on the held-out 20% (Step 7), saved to `traffic_model.pkl` using Joblib (Step 10), reloaded, and used to generate predictions from user-supplied Junction and ID values through a command-line interface (Steps 11–12).

6.3 Performance Outcome

The correlation analysis showed that Junction has a moderate negative correlation with Vehicles (≈ -0.61), meaning traffic volume tends to be lower at higher-numbered junctions. Junction and ID both show a weak positive correlation with Vehicles (≈ 0.22 and 0.23 respectively).

Metric	Value
Training records	48,120
Train / Test split	80% / 20% (random_state = 42)
Mean Absolute Error (MAE)	≈ 10.33 vehicles
Mean Squared Error (MSE)	≈ 198.95
Root Mean Squared Error (RMSE)	≈ 14.11 vehicles

The bar chart of average traffic per junction (Step 9) shows a clear difference between junctions: Junction 1 carries by far the highest average traffic (≈ 45 vehicles/hour), while Junctions 2, 3 and 4 carry progressively less (≈ 14.3 , 13.7 and 7.3 vehicles/hour respectively). This matches the negative Junction–Vehicles correlation observed in the heatmap and indicates that Junction number is a meaningful, informative feature for the model.

Overall, given that only two simple features (Junction and ID) were used, the Linear Regression model achieves a reasonable baseline accuracy. Since ID is essentially a numeric encoding of the timestamp, it carries some time information indirectly, but not as effectively as directly using Year, Month, Day and Hour — which were engineered in Step 3 but not carried into the final model used in Steps 6–12. This is noted as a key area of improvement in the Future Scope section.

7. My Learnings

This internship gave me practical, end-to-end exposure to a machine learning workflow rather than just isolated concepts. Working through each step individually helped me build a clearer mental model of how a real project is structured:

- Learned to load and explore tabular data using Pandas — checking shape, column types, missing values, and summary statistics before doing anything else.
- Learned to convert string timestamps into proper datetime objects and extract meaningful time-based features (Year, Month, Day, Hour).
- Learned to visualise data effectively using Matplotlib — line plots for trends, a correlation heatmap for relationships between variables, and bar charts for group-wise comparisons.
- Learned to train, evaluate (MAE, MSE) and interpret a Linear Regression model using scikit-learn.
- Understood the importance of selecting appropriate features and maintaining consistency throughout the machine learning pipeline to improve model performance and reliability.
- Learned to persist a trained model using Joblib and reload it in a separate script for reuse, and to build a simple command-line interface around it.

Beyond the technical skills, this internship also helped me build the discipline of breaking a large, ambiguous problem statement into small, testable, sequential steps — a habit I intend to carry forward into future projects and my career.

8. Future Work Scope

- Retrain the final model using the engineered Year, Month, Day and Hour features (already created in Step 3) instead of only Junction and ID, which should improve prediction accuracy.
- Experiment with other regression algorithms such as Random Forest Regressor, Gradient Boosting, or XGBoost, and compare their MAE/MSE against the current Linear Regression baseline.
- Explore dedicated time-series approaches (e.g., ARIMA, SARIMA, or Facebook Prophet) since traffic volume is fundamentally a time-series signal with daily and weekly seasonality.
- Incorporate additional context such as public holidays and special events, which the original problem statement specifically calls out as affecting traffic patterns.
- Wrap the saved model in a small web API (e.g., using Flask or FastAPI) so predictions can be requested over HTTP instead of only through a command-line prompt.
- Extend the visualisation layer into an interactive dashboard so city planners can explore predicted traffic at each junction over time.

9. Project Screenshots

9.1 Terminal Output

This section is reserved for the console/terminal output produced while running Steps 1–2 (dataset loading and exploration) and Step 4 (traffic trend plot). The screenshot could not be included here due to the upload size limit and will be inserted before submission.

```
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns - Copy\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\smart-city-traffic-patterns> python step1_load_data.py
Training Data Shape: (48120, 4)
Test Data Shape: (11808, 3)

First 5 rows of Training Data:
   DateTime Junction Vehicles      ID
0 2015-11-01 00:00:00      1     15 20151101001
1 2015-11-01 01:00:00      1     13 20151101011
2 2015-11-01 02:00:00      1     10 20151101021
3 2015-11-01 03:00:00      1      7 20151101031
4 2015-11-01 04:00:00      1      9 20151101041
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns - Copy\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\smart-city-traffic-patterns>
```

Figure 4: Terminal Output – Dataset Loading (Step 1)

```
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns - Copy\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\smart-city-traffic-patterns> python step2_explore_data.py
Dataset Shape: (48120, 4)

Columns:
Index(['DateTime', 'Junction', 'Vehicles', 'ID'], dtype='object')

Missing Values:
DateTime    0
Junction    0
Vehicles    0
ID          0
dtype: int64

Statistics:
   Junction  Vehicles      ID
count  48120.000000  48120.000000  4.812000e+04
mean     2.180549    22.791334  2.016330e+10
std      0.966955    20.750063  5.944854e+06
min      1.000000     1.000000  2.015110e+10
25%      1.000000     9.000000  2.016042e+10
50%      2.000000    15.000000  2.016093e+10
75%      3.000000    29.000000  2.017023e+10
max      4.000000   180.000000  2.017063e+10

First 5 Rows:
   DateTime Junction Vehicles      ID
0 2015-11-01 00:00:00      1     15 20151101001
1 2015-11-01 01:00:00      1     13 20151101011
2 2015-11-01 02:00:00      1     10 20151101021
3 2015-11-01 03:00:00      1      7 20151101031
4 2015-11-01 04:00:00      1      9 20151101041
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns - Copy\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\smart-city-traffic-patterns>
```

Figure 5: Terminal Output – Data Exploration (Step 2)

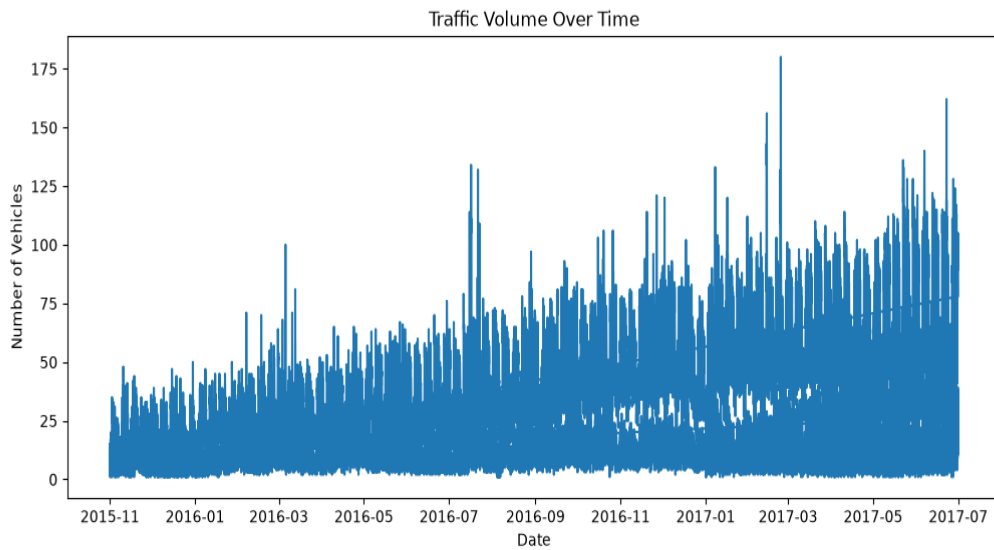


Figure 6: Traffic Volume Over Time (Step 4)

9.2 Data Preprocessing

This figure shows the data preprocessing stage of the project. During this step, the DateTime column was converted into datetime format, and important features such as Year, Month, Day, and Hour were extracted. These preprocessing operations prepared the dataset for machine learning model training and improved the quality of the input data.

```
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns - Copy\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns> python step3_preprocessing.py
```

	DateTime	Junction	Vehicles	ID	Year	Month	Day	Hour
0	2015-11-01 00:00:00	1	15	20151101001	2015	11	1	0
1	2015-11-01 01:00:00	1	13	20151101011	2015	11	1	1
2	2015-11-01 02:00:00	1	10	20151101021	2015	11	1	2
3	2015-11-01 03:00:00	1	7	20151101031	2015	11	1	3
4	2015-11-01 04:00:00	1	9	20151101041	2015	11	1	4

```
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns - Copy\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns>
```

Figure 7: Data Preprocessing and Feature Engineering (Step 3).

9.3 Correlation Heatmap

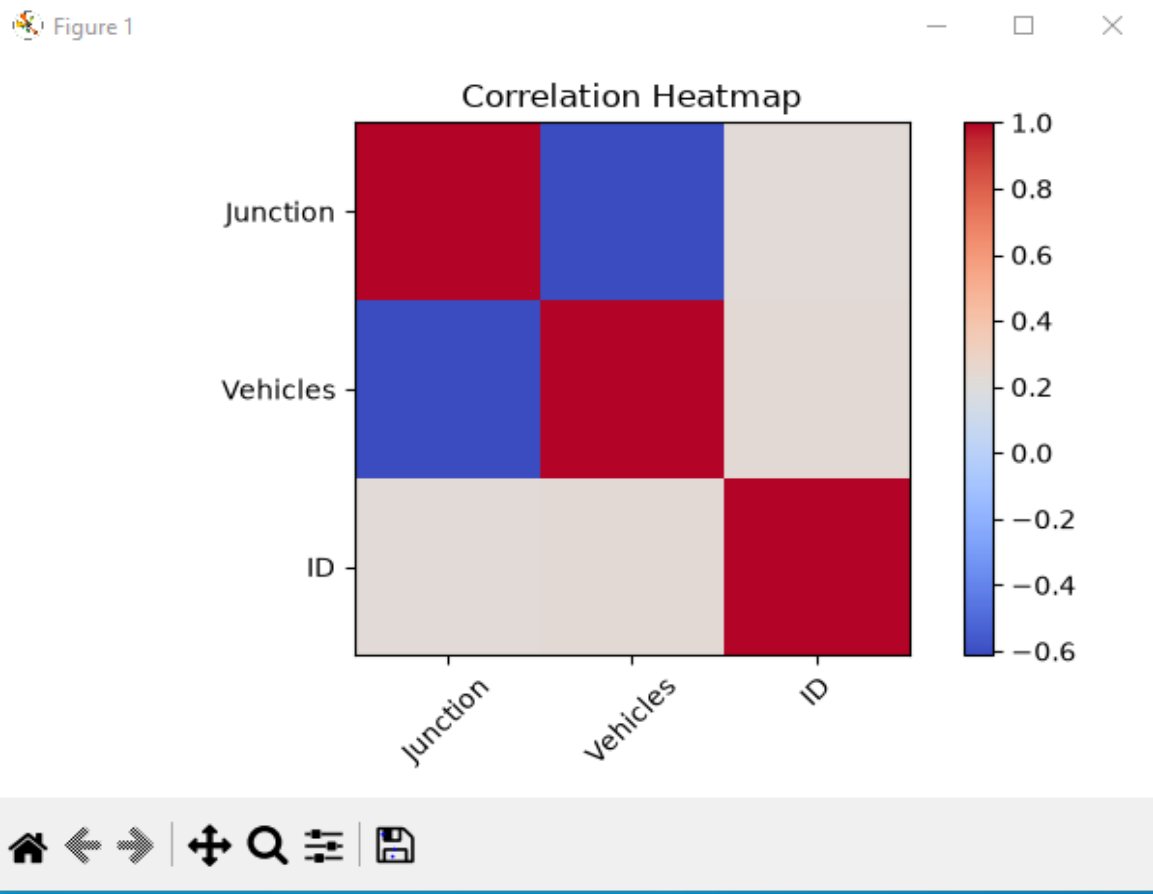


Figure 8: Correlation heatmap of Junction, Vehicles and ID (Step 5).

9.4 Average Traffic at Junctions

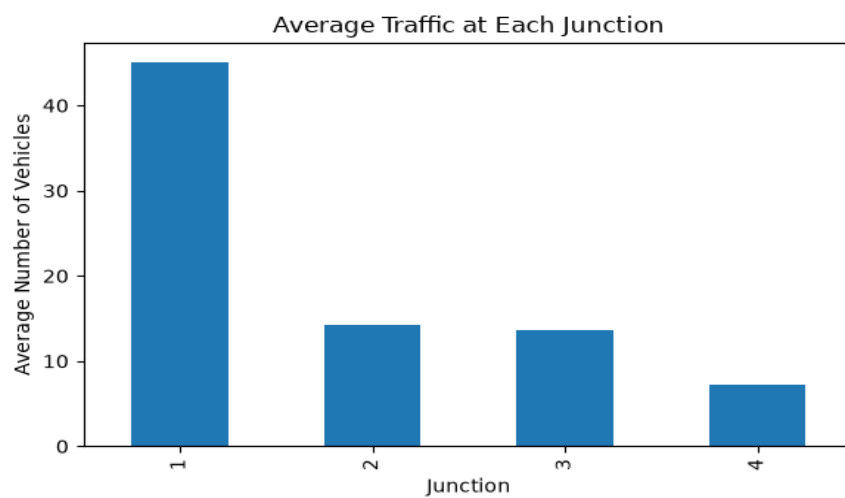


Figure 9: Average traffic at each junction (Step 9).

9.5 Model Evaluation Output (MAE & MSE)

This figure shows the evaluation results of the trained Linear Regression model. The model's performance was measured using Mean Absolute Error (MAE) and Mean Squared Error (MSE). These metrics were used to evaluate the prediction accuracy of the model on the test dataset.

```
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns - Copy\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\smart-city-traffic-patterns> python step7_evaluation.py
Mean Absolute Error: 10.332873287602471
Mean Squared Error: 198.9537263021777
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns - Copy\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\smart-city-traffic-patterns>
```

Figure 10: Model Evaluation Output (MAE and MSE)

9.6 Model Saved

The trained Linear Regression model was successfully saved using the Joblib library. Saving the trained model allows it to be reused for future predictions without retraining the model, improving efficiency and enabling easy deployment.

```
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\smart-city-traffic-patterns> python step10_save_model.py
Model saved successfully as traffic_model.pkl
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\smart-city-traffic-patterns>
```

Figure 11: Model Saved

9.7 Final Prediction Output

This section is reserved for the console output of Step 12, showing the final predicted traffic value returned by the Smart City Traffic Prediction System for a user-entered Junction and ID. The screenshot could not be included here due to the upload size limit and will be inserted before submission.

```
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\smar
t-city-traffic-patterns> python step12_final_project.py
=== Smart City Traffic Prediction System ===
Enter Junction Number: 1
Enter ID: 2017110101

Predicted Traffic: -24218.807697120064
Prediction completed successfully!
PS C:\Users\mi\Downloads\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\Project9_smart-city-traffic-patterns\smar
t-city-traffic-patterns> 
```

Ln 19, Col 55 Spaces: 4 UTF-8 CRLF () Python Signed out 3.13.7 Python 3.13 (64-bit) Go Live Prettier

Figure 12: Final Prediction Output.

10. GitHub Repository

The complete source code for this project (Step 1 through Step 12), along with the datasets and this report, has been uploaded to GitHub.

Repository Link: <https://github.com/shalini05gupta11-cmd/upskillCampus>

Project Report Link: [https://github.com/shalini05gupta11-cmd/upskillCampus/blob/main/Internship_Report_Smart_City_Traffic%20\(1\)new2.docx](https://github.com/shalini05gupta11-cmd/upskillCampus/blob/main/Internship_Report_Smart_City_Traffic%20(1)new2.docx)

11. Skills Used

- Python
- Pandas — data loading, cleaning, and manipulation
- Data Analysis — dataset exploration, shape/column/missing-value checks, statistical summaries
- Data Preprocessing — DateTime parsing and cleaning
- Feature Engineering — extracting Year, Month, Day and Hour from timestamps
- Machine Learning — supervised regression modelling
- Linear Regression
- scikit-learn — model training, train/test split, evaluation metrics
- Model Evaluation — Mean Absolute Error (MAE), Mean Squared Error (MSE)
- Data Visualization — Matplotlib (line plots, heatmaps, bar charts)
- Model Persistence — Joblib (saving and loading trained models)
- GitHub